

1. **What are the differences between Native, Hybrid, and Cross-platform mobile application development, and when is each approach commonly used?**

There are three common ways to build mobile applications: Native, Hybrid, and Cross-platform, based on my research.

Native Development

Native apps are made for only one operating system, either Android or iOS.

- Android apps use Kotlin or Java.
- iOS apps use Swift.

Because they are built only for one platform, they are fast, smooth, and can use all the phone's features, such as the camera, GPS, and Bluetooth.

Common uses:

- Mobile games
- Banking apps
- Camera apps
- Apps that need high performance

Hybrid Development

Hybrid apps are built using HTML, CSS, and JavaScript. These are web technologies, but the app is placed inside a native app so it can run on phones.

Hybrid apps are faster to build, but they may be slower than native apps.

Common uses:

- Business apps
- School projects
- Simple company apps
- Quick prototypes

Cross-platform Development

Cross-platform apps use one code that works on both Android and iOS. Popular frameworks are Flutter and React Native.

This saves time because developers do not need to write two separate applications.

Common uses:

- E-commerce apps
- School projects

- Startup apps
 - Apps that need to work on Android and iOS with less time and lower cost
2. **What is the Flutter Ecosystem, and how does it help developers create modern mobile applications?**

So, based on the research that was given to me, the Flutter Ecosystem is the collection of tools that developers use to build mobile applications with Flutter.

It includes:

- **Flutter Framework** – provides ready-made widgets to create beautiful user interfaces.
- **Dart** – the programming language used in Flutter.
- **Flutter DevTools** – helps developers test, fix bugs, and improve app performance.
- **Packages and Plugins** – ready-made features from pub.dev, such as login, camera, maps, and databases.

The Flutter Ecosystem helps developers build apps faster, easier, and with one code for both Android and iOS.

3. **What is the Software Development Life Cycle (SDLC), and why is it important in building successful applications?**

The Software Development Life Cycle (SDLC) is the step-by-step process used to create software or mobile applications, as shown in the research.

These are the main stages:

- **Planning**
- **Analysis**
- **Design**
- **Coding (Implementation)**
- **Testing**
- **Deployment and Maintenance**

The SDLC is important because it helps developers stay organized. It also reduces mistakes, improves software quality, saves time, and makes sure the application meets the users' needs.

4. **How do Flutter development practices and the SDLC work together in creating real-world mobile applications?**

Flutter and the SDLC work together during the whole app development process.

First, developers use the SDLC to plan and design the application. Then they use Flutter to build the user interface and write the code using Dart.

After coding, Flutter provides testing tools to find and fix problems. When everything is working correctly, the app is released for Android and iOS.

By using Flutter together with the SDLC, developers can build applications that are organized, reliable, and easy to maintain.

5. **Why should developers understand these concepts before starting an application project?**

Developers should understand these concepts before starting a project because they help them make better decisions.

Knowing the difference between Native, Hybrid, and Cross-platform helps developers choose the best way to build an application.

Understanding the Flutter Ecosystem helps them develop apps faster using one code for different platforms.

Learning the SDLC helps them follow the correct process from planning to maintenance.

When developers understand these concepts, they can save time, reduce errors, build better applications, and meet the needs of users more effectively.